Dimensionality Reduction Visualization

Workshop 5 — Module 3

Prof.Asoc. Endri Raco

2025/26

Data Visualization for Data Scientists
UNYT Tirana

Module Roadmap

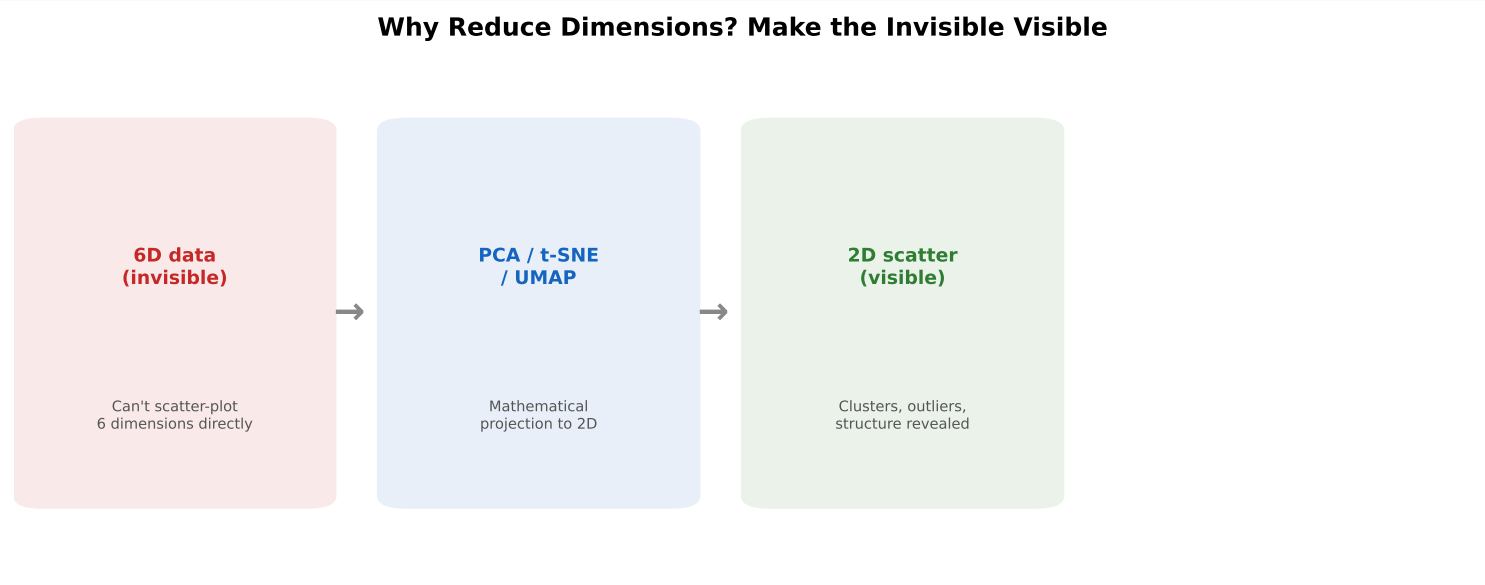
1. Why reduce: making the invisible visible
2. PCA: linear projection onto maximum-variance axes
3. The scree plot: how many components to keep
4. The biplot: scores + loadings in one figure
5. t-SNE: nonlinear, local structure preservation
6. UMAP: nonlinear, global + local topology

Learning Outcome

You will interpret PCA biplots (loading arrows = variable contribution), produce scree plots to choose the number of components, and compare PCA / t-SNE / UMAP projections to understand their different preservation properties.

Why Reduce?

Make the Invisible Visible

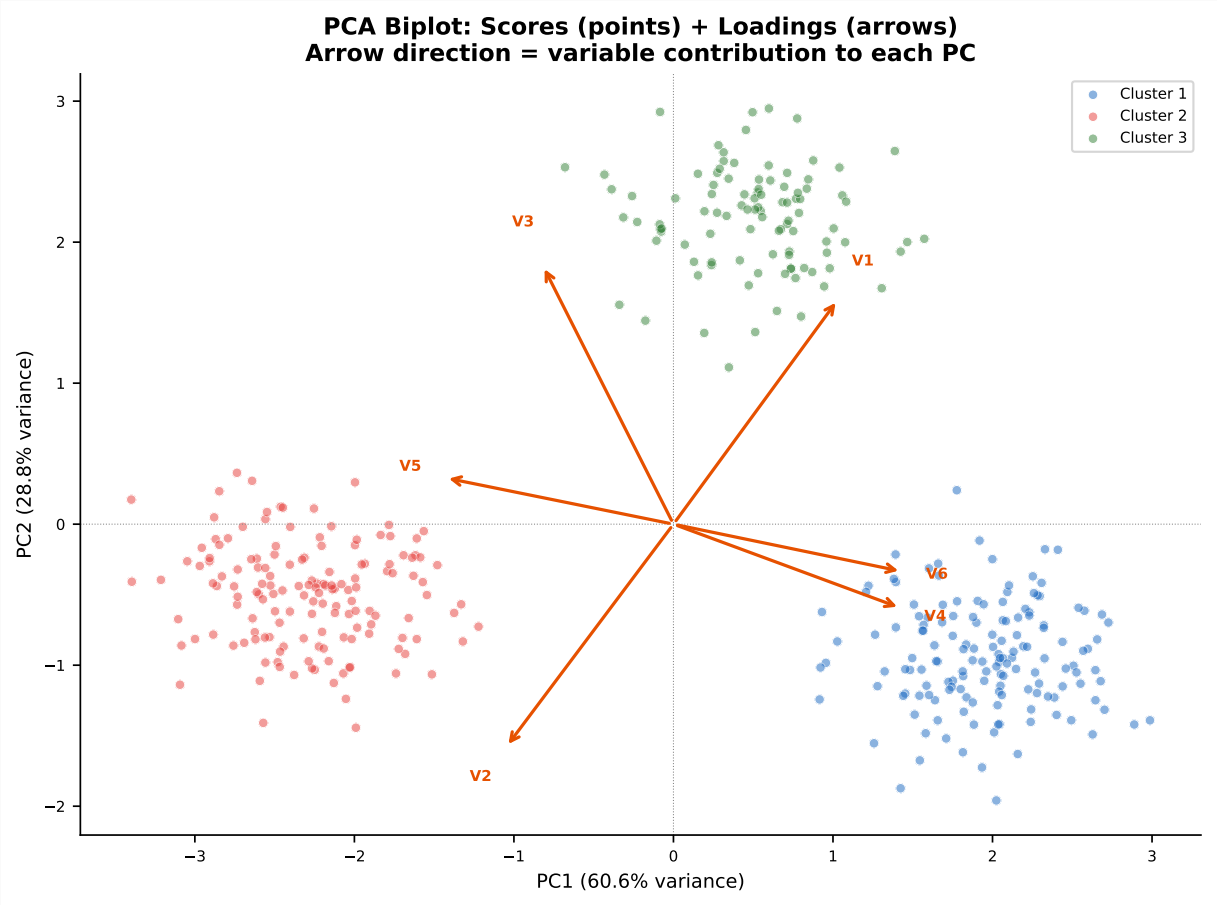


Pro-Tip

Dimension reduction is **not** about discarding information. It is about finding the 2D view that preserves the most important structure (variance for PCA, neighbourhoods for t-SNE, topology for UMAP). Think of it as choosing the best camera angle for a 3D object.

PCA

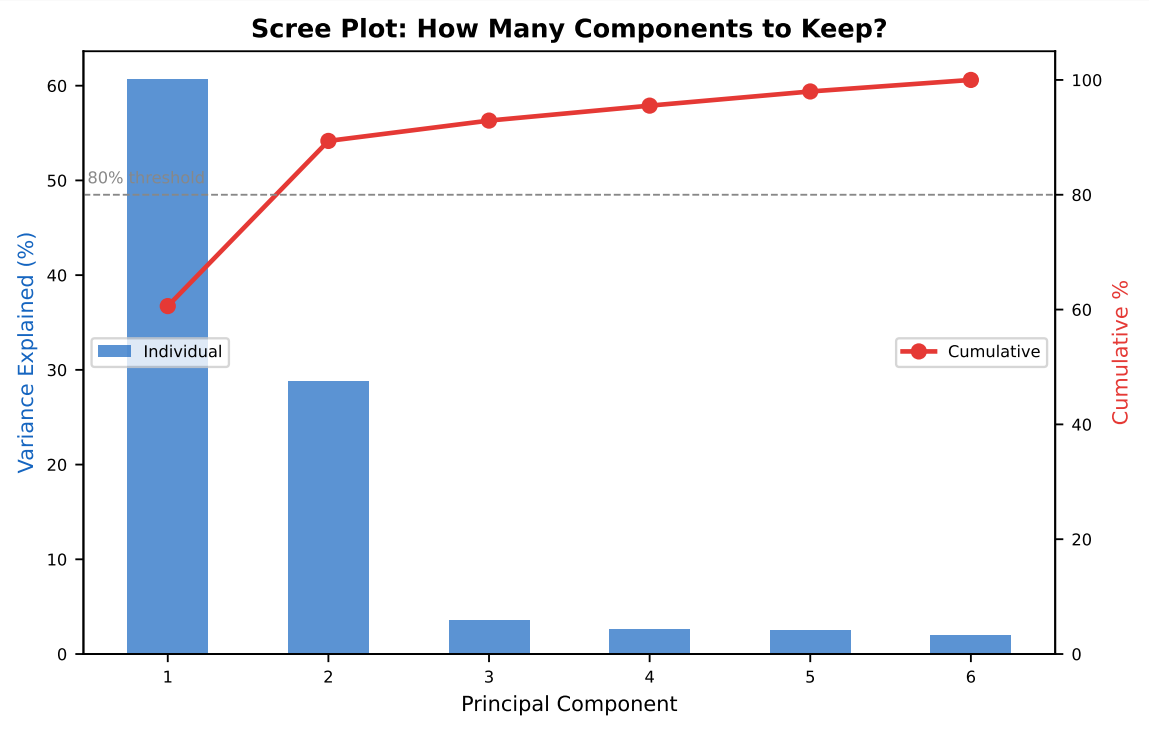
PCA: Linear Projection onto Max-Variance Axes



How to Read a Biplot

Element	Interpretation
Points (scores)	Observations projected into PC space. Close points = similar multivariate profiles.
Arrows (loadings)	Original variables. Arrow direction = which PC the variable contributes to. Arrow length = strength of contribution.
Angle between arrows	$\cos(\theta) \approx$ correlation. Parallel = positive corr. Opposite = negative. Perpendicular = uncorrelated.
Distance from origin	Extremity. Points far from centre are unusual in PC space (potential outliers).
% variance on axes	How much of total variance this view captures. PC1+PC2 $>$ 70% = good 2D summary.

Scree Plot: How Many Components?



Data Insight

Two rules of thumb: (1) **Elbow rule**: keep components before the scree "flattens." (2) **80% rule**: keep enough components to explain $\geq$80% of cumulative variance. Here, 2–3 components suffice for the 6D data.

PCA in Code

R

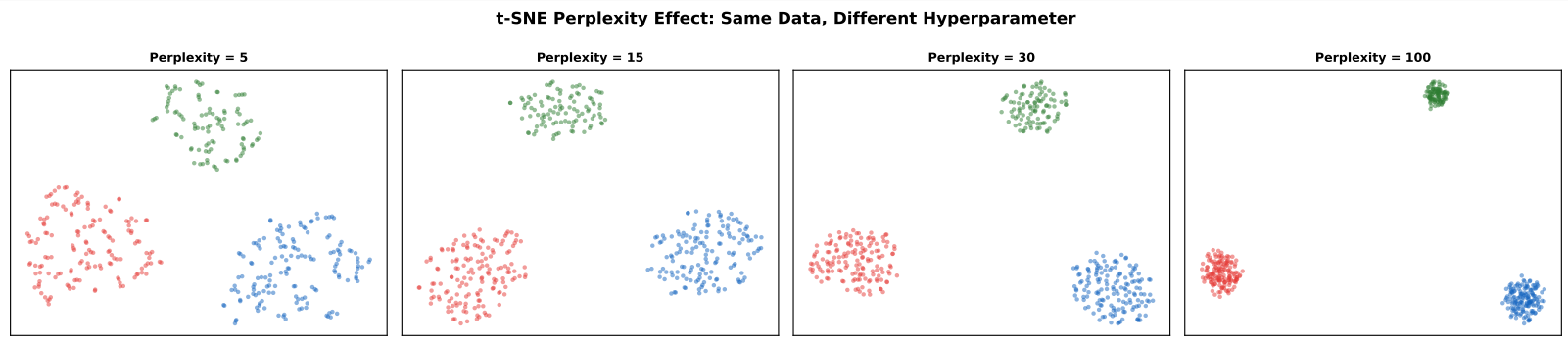
```r
# PCA
pca <- prcomp(df[, num_cols],
  scale. = TRUE)  # always scale!

# Scree
summary(pca)  # proportion of var

# Quick biplot
biplot(pca)   # base R

# ggplot biplot (ggfortify)
library(ggfortify)
autoplot(pca,
  data = df,
  colour = "Cluster",
  loadings = TRUE,
  loadings.label = TRUE,
  loadings.colour = "#E65100") +
  theme_minimal()
```

Python

```python
from sklearn.decomposition import
     PCA
from sklearn.preprocessing import
     (
    StandardScaler)

X_scaled = StandardScaler().
     fit_transform(
    df[num_cols])
pca = PCA(n_components=2)
scores = pca.fit_transform(
     X_scaled)

# Scree
var_exp = pca.
     explained_variance_ratio_

# Biplot (manual)
fig, ax = plt.subplots()
ax.scatter(scores[:,0], scores
     [:,1],
    c=labels, cmap="viridis", s
         =15)
# Loading arrows
```

t-SNE

t-SNE: Nonlinear, Local Structure



Pro-Tip

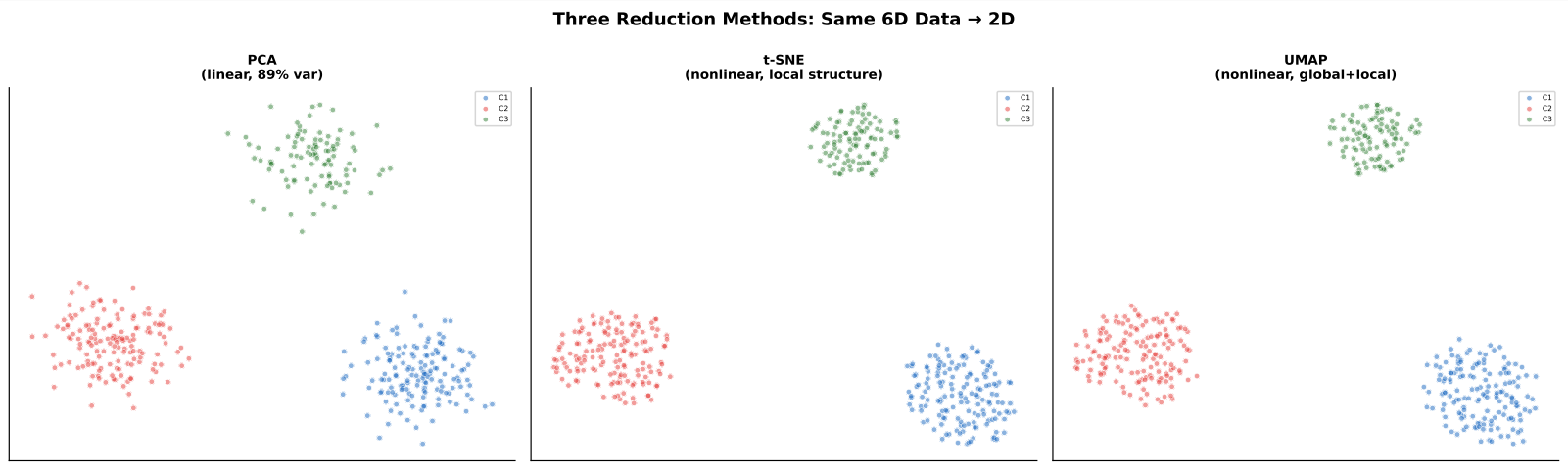
t-SNE (van der Maaten & Hinton, 2008) preserves **local** neighbourhoods: nearby points stay nearby. But **distances between clusters are meaningless** — cluster size and gap width are artefacts. The **perplexity** hyperparameter ($\sim$5–50) controls the effective neighbourhood size. Always try 2–3 values.

t-SNE: What You Can and Cannot Conclude

Conclusion	Valid?	Why
"These points form a cluster"	✓	Local structure preserved
"Cluster A is closer to B than C"	×	Inter-cluster distances arbitrary
"Cluster A is larger than B"	×	Size is an artefact of density
"This point is an outlier"	✓	Isolated points = dissimilar
"PC1 correlates with variable X"	×	No axes interpretation (unlike PC

UMAP

PCA vs t-SNE vs UMAP: Same Data, Three Views



t-SNE and UMAP in Code

R

```r
# t-SNE
library(Rtsne)
tsne <- Rtsne(X_scaled,
  dims = 2, perplexity = 30,
  check_duplicates = FALSE)
plot(tsne$Y, col = labels, pch =
    16)

# UMAP
library(umap)
um <- umap(X_scaled)
plot(um$layout, col = labels, pch
    =16)

# ggplot wrapper
tibble(x = tsne$Y[,1],
       y = tsne$Y[,2],
       cluster = factor(labels))
           |>
  ggplot(aes(x, y, color =
      cluster)) +
  geom_point(alpha = 0.5, size =
      1) +
  theme_minimal() +
```

Python

```python
# t-SNE
from sklearn.manifold import TSNE
tsne = TSNE(n_components=2,
    perplexity=30, random_state
        =42)
X_tsne = tsne.fit_transform(
    X_scaled)
plt.scatter(X_tsne[:,0], X_tsne
    [:,1],
    c=labels, cmap="viridis", s
        =10)

# UMAP
# pip install umap-learn
from umap import UMAP
um = UMAP(n_components=2,
    random_state=42)
X_umap = um.fit_transform(
    X_scaled)
plt.scatter(X_umap[:,0], X_umap
    [:,1],
    c=labels, cmap="viridis", s
        =10)
```

Choosing a Method

PCA vs t-SNE vs UMAP

PCA vs t-SNE vs UMAP: When to Use Which

Method	How	Preserves	Axes Meaning	Speed
PCA	Linear projection onto max-variance axes	Global structure preserved	Distances meaningful → biplot interpretable	Fast ($O(np^2)$) Deterministic
t-SNE	Nonlinear: preserve local neighbourhoods	Local clusters best separated	Distances NOT meaningful → cluster size/gap arbitrary	Slow ($O(n^2)$) Stochastic
UMAP	Nonlinear: preserve local + global topology	Both local and global structure	Faster than t-SNE better at preserving global	Fast ($O(n \log n)$) Stochastic

Decision Rules

Goal	Method	Why
Understand variable contributions	PCA	Loadings are interpreta
Find the "best 2D summary"	PCA	Max-variance projectio
Discover clusters visually	t-SNE or UMAP	Nonlinear separation
Large dataset (>50K rows)	UMAP	$O(n \log n)$ vs $O(n^2)$
Need reproducibility	PCA	Deterministic
Preserve global distances	PCA or UMAP	t-SNE distorts global

Pro-Tip

Best practice: start with PCA (interpretable, fast), check the scree plot. If the first 2 PCs capture <50% variance, the data has complex nonlinear structure — then try t-SNE or UMAP. Always show PCA alongside t-SNE/UMAP so the reader can compare.

Common Pitfalls

Five Pitfalls of Dimension Reduction

#	Pitfall	Solution
1	Forgetting to scale	Always `scale. = TRUE` (R) or `StandardScaler()` (Python)
2	Over-interpreting t-SNE distances	State: "distances between clusters are not meaningful"
3	Single perplexity / n_neighbors	Try 3+ values; show in a panel
4	Treating PCs as "real" variables	PCs are combinations; interpret via loadings
5	Running t-SNE on >50K rows	Use UMAP or subsample first

Synthesis

Key Takeaways

1. **PCA** = linear, interpretable (biplot + scree), deterministic. Use when you need to understand *which variables drive* the structure. Always scale first.
2. **Scree plot**: elbow rule or 80% cumulative variance to choose the number of components.
3. **Biplot**: points = observations, arrows = variable contributions, angle between arrows $\approx$ correlation.
4. **t-SNE**: nonlinear, excellent at revealing clusters. But distances between clusters are meaningless; always report perplexity.
5. **UMAP**: nonlinear like t-SNE but faster and better at preserving global structure. Preferred for large datasets.
6. **Best practice**: PCA first (interpretable baseline), then t-SNE/UMAP if nonlinear structure suspected.

Essential Readings

- Jolliffe, I. T. (2002). *Principal Component Analysis*, 2nd ed. Springer.
- van der Maaten, L. & Hinton, G. (2008). "Visualizing Data using t-SNE." *JMLR*, 9, 2579–2605.
- McInnes, L. et al. (2018). "UMAP: Uniform Manifold Approximation and Projection." arXiv:1802.03426.
- Wattenberg, M. et al. (2016). "How to Use t-SNE Effectively." *Distill*. `https://distill.pub/2016/misread-tsne/`

Next Module: Heatmaps & Clustered Displays (W05-M04)